

Practica Integradora Final en el Proceso de Desarrollo de Software

Objetivo General

Este trabajo práctico integrador tiene como finalidad evaluar el manejo integral de herramientas avanzadas aplicadas al ciclo de vida del software: implementación, despliegue, mantenimiento, gestión de incidencias, monitoreo de infraestructura, optimización de bases de datos y aplicación de inteligencia artificial.

El proyecto deberá demostrar la capacidad de los equipos para integrar tecnologías modernas y garantizar la calidad, seguridad y escalabilidad de una aplicación web en un entorno de desarrollo y producción controlado.

Plan de Acción

1. Selección de la Aplicación Web

Tarea: Seleccionar una aplicación web previamente desarrollada (por ejemplo, un proyecto académico o institucional).

Identificar el stack tecnológico utilizado:

- Backend: Node.js, Spring Boot, Django u otro.
- Frontend: Angular, React, Vue, etc.
 - Debe incluir **principios UX/UI y analítica de datos**.
- Base de datos: PostgreSQL o MySQL/MariaDB.

Si la aplicación no se encuentra en un repositorio remoto, inicializarlo con Git y publicarlo en GitHub o GitLab.

2. Gestión de Proyectos

- Implementar una metodología ágil basada en Scrum, definiendo Sprints, backlog y responsables.
- Herramienta recomendada: Jira o equivalente.

3. Contenedores para Servicios y Base de Datos

Objetivo: contenerizar todos los servicios para aislar dependencias y simplificar despliegues.

- Crear un contenedor para la base de datos (PostgreSQL o MySQL).
- Configurar un cliente dockerizado (pgAdmin, phpMyAdmin, etc.).
- Migrar los datos existentes al contenedor.
- Configurar backups automáticos mediante un contenedor adicional.
- Incorporar dataset de prueba (mínimo 500.000 registros por tabla) para ejecutar tareas de optimización y análisis.
- Aplicar herramientas de optimización (pgBadger, MySQLTuner, Percona Toolkit, etc.).

4. Contenedores para la Aplicación Web

Dockerizar la aplicación principal (frontend y backend) y exponerla al navegador. Cada componente debe correr en su contenedor:

- Frontend: Angular/React/Vue.
- Backend: Node.js/Spring/Django.

Integrar herramientas de monitoreo y visualización como Prometheus y Grafana para mostrar métricas en tiempo real (CPU, memoria, red, uso de contenedores, consultas SQL, etc.).

El dashboard debe permitir la observación del rendimiento general del sistema.

5. Despliegue continuo

Automatizar el flujo completo de integración y entrega continua (CI/CD):

- Herramientas recomendadas: GitHub Actions, GitLab CI/CD o Jenkins.
- El pipeline debe detectar cambios (push o merge) en el repositorio, reconstruir la imagen del contenedor y desplegar automáticamente.
- Modificar el header o footer de la aplicación para incluir los nombres de los integrantes.
- Validar el despliegue automático con un commit de prueba.

6. Clúster de Replicación

Configurar un clúster de base de datos con replicación maestro-esclavo o multi-nodo:

- Configurar la aplicación para apuntar al clúster.
- Ejecutar pruebas de replicación, alta disponibilidad y recuperación ante fallos.
- Documentar la configuración y resultados.

7. Gestión de Incidencias

Implementar un sistema de registro, seguimiento y control de errores:

- Herramienta recomendada: Mantis Bug Tracker o Glpi
- Registrar incidencias detectadas durante desarrollo y pruebas.
- Priorizar y asignar las tareas al responsable correspondiente.

8. Gestión Documental (NextCloud)

Utilizar NextCloud para almacenar y compartir documentación técnica, reportes, respaldos de configuración y registros del proyecto.

Opcional: Se debe además integrar un módulo de IA que analice la analítica de datos generada y produzca informes automáticos de desempeño.

9. Seguridad

Implementar medidas de seguridad y defensa activa:

- Firewall (UFW / iptables).
- Scripts de mitigación DDoS.
- Políticas de autenticación, cifrado SSL/TLS y control de accesos.
- Informe final con pruebas simuladas de ataque y mitigación exitosa

Tabla de Tecnologías

Etapas	Herramienta / Tecnología	Descripción
Planificación	Jira	Gestión ágil con Sprints y backlog.
Control de versiones	Git + GitHub	Repositorio principal con ramas feature/dev/main.
Integración / Entrega	GitHub Actions / GitLab CI/CD	Automatización de build, test y despliegue.
Despliegue	Docker + Docker Compose	Contenerización de backend, DB y frontend.
Base de Datos	MariaDB Cluster / PostgreSQL Cluster	Alta disponibilidad, replicación y optimización.
Testing	Mantis Bug Tracker o GLPI	Registro de errores e incidencias.
Documentación	NextCloud	Centralización de entregables y reportes.
Seguridad	Firewall + Scripts DDoS	Control de acceso y defensa ante ataques.
Inteligencia Artificial	Python + Scikit-learn / TensorFlow	Detección de anomalías y generación de informes.

Lineamiento y Recomendaciones

- **Equipos:** 5 integrantes.
- **Presentación:** cada miembro debe participar activamente durante la exposición.
- **Versionamiento:** mantener un flujo de trabajo limpio con ramas (feature/, develop/, main) y commits descriptivos.
- **Plazo de inscripción:** hasta el **lunes 3 de noviembre**.
- **Presentación final:** semana del **21 de noviembre**.

Evaluación

- La **nota será individual**, aunque el proyecto sea grupal.
- Se aprueba con **nota igual o superior a 7 (siete)**.
- Cada integrante deberá demostrar comprensión técnica de los componentes implementados.
- Se valorará:
 - o Integración correcta de todas las herramientas.
 - o Calidad del despliegue CI/CD.
 - o Correcta configuración del clúster de BD.
 - o Funcionamiento del monitoreo y de la IA.
 - o Nivel de documentación técnica y claridad en la exposición.

Entregables

1. **Documentación Técnica:**
 - o Arquitectura del sistema.
 - o Manual de instalación y configuración Docker + CI/CD.
2. **Repositorio GitHub Público:**
 - o Código fuente organizado, ramas, commits y README.
3. **Presentación (10-15 min):**
 - o Roles, funcionamiento del sistema, CI/CD y demostración en vivo.
4. **Matriz de Evaluación de Herramientas:**
 - o Comparativa de las herramientas utilizadas (Docker, Jira, Mantis, NextCloud, etc.).
5. **Informe de IA y Seguridad:**
 - o Resultados de detección de anomalías.
 - o Pruebas simuladas de ataque DDoS y mitigación.